Phase 1 Study Notes — AI Engineering Self-Study

Comprehensive notes compiled from tutor sessions covering Weeks 1–4: Modern Python, LLM APIs, Embeddings & Vector Search, and RAG.

Week 1: Modern Python Reset

1.1 Type Hints

What they are

Annotations on variables and functions declaring expected types. Python doesn't enforce them at runtime — tools like `mypy`, `pyright`, and IDEs use them for static checking, autocomplete, and bug detection.

Why they matter

- Self-documenting code
- Catch bugs before runtime via static checkers
- Foundation for Pydantic, FastAPI, every modern Python AI library

Key syntax

Form	Meaning
`name: str`	parameter type
`-> str`	return type
`list[str]`, `dict[str, int]`, `tuple[int, str]`	collection generics (Python 3.9+)
`str | None`	union type (Python 3.10+, replaces `Optional[str]`)
`Callable[[str], str]`	function type — args list, then return
`TypedDict`	dict with per-key types
`type[BaseModel]`	a class (not instance)

Examples written

```
def greet(name: str) -> str:
    return f"Hello {name}"

def formal_greeting(name: str, title: str | None = None) -> str:
    if title is None:
        return f"Hello, {name}"
    else:
        return f"Hello, {title} {name}"

class WordSummary(TypedDict):
    count: int
    longest: str

def summarize(words: list[str]) -> WordSummary:
    count = len(words)
    max_len = 0
    longest = ""
    for word in words:
        word_length = len(word)
        if word_length > max_len:
            longest = word
            max_len = word_length
    return {"count": count, "longest": longest}

def transform_words(words: list[str], func: Callable[[str], str]) -> list[str]:
    return [func(word) for word in words]
```

Q&A

Q: Why does `greet(42)` still work if I declared `name: str`? Type hints are not enforced at runtime. Python doesn't check them. Static tools (mypy/pyright/IDE) flag the mismatch, but the interpreter ignores them. Hints = documentation + tooling, NOT runtime protection. Pydantic exists precisely to fill this gap with runtime validation.

Q: What is a tuple? Fixed-length, ordered, immutable collection. Like a list, but cannot be resized or modified after creation. Use for multiple related return values that always come together (e.g., `(chunk, score)`). `tuple[int, str]` means exactly two items of those types.

Q: Why is shadowing built-ins (e.g. `max = 0`) bad? Inside that scope, `max` no longer refers to the built-in function. Later calls to `max(some_list)` fail silently or behave unexpectedly. Use names like `max_len` or `current_max`.

1.2 Pydantic v2

What it is

A library that enforces types **at runtime** by defining models as classes. Bad data fails loudly at the boundary instead of silently corrupting downstream logic.

Why it matters

- Validates LLM outputs (the LLM said JSON — did it really?)
- Defines API request/response schemas
- Used internally by FastAPI, LangChain, every LLM framework

Key features

- **Validation** — wrong types raise `ValidationError`
- **Coercion** — `"53"` becomes `53` automatically when field is `int`. `"ab"` to `float` fails, because there is no sensible conversion
- **`Field()`** — adds constraints (`gt`, `lt`, `ge`, `le`) and metadata
- **`model_validate_json()`** — parse + validate a raw JSON string in one call (killer feature for LLM output)

Examples written

```
class Book(BaseModel):
    title: str
    pages: int = Field(gt=0)
    rating: float | None = Field(default=None, ge=0, le=5)

# Works (coerces "53" -> 53)
Book(title="ABC", pages="53", rating=2.2)

# Fails: rating out of range
Book(title="ABC", pages=53, rating=-5)

# Parses an LLM JSON output directly
Book.model_validate_json('{"title":"ABC","pages":52,"rating":2.2}')
```

How `TypedDict` differs from `BaseModel`

`TypedDict` is just a type hint — no runtime enforcement. `BaseModel` validates at runtime. Use `TypedDict` for static type-checking of plain dicts, `BaseModel` everywhere you receive external data.

Q&A

Q: Why use keyword arguments with Pydantic models? `Book("ABC", 53, 3.1)` fails with `TypeError`: BaseModel doesn't accept positional args. Always pass field names explicitly.

Q: Field names matter for LLM structured outputs — why? When you pass `response_format=Sentiment` to OpenAI, the SDK sends the Pydantic schema (including field names) to the model. The model uses field names to infer what values to put. `label`, `confidence`, `reasoning` communicate intent. A field called `x` would leave the model guessing.

1.3 Async

What it is

A way for Python to do other work while waiting for slow I/O (like API calls). Lets you fire many slow calls in parallel and wait for them together.

Three rules

1. `async def` defines a coroutine — a pauseable function
2. `await` is the pause point — only valid inside `async def`
3. `asyncio.gather(*tasks)` runs many coroutines in parallel

Power Automate analogy

Parallel branches that join at the end. `asyncio.gather` IS the join.

Examples written

```
async def fetch_summary(name: str) -> str:
    await asyncio.sleep(1)
    return f"Summary for {name}"

async def main() -> None:
    result = await asyncio.gather(
        fetch_summary("ABC"),
        fetch_summary("DEF"),
        fetch_summary("GHI"),
    )
    print(result)

await main()  # ~1 second total, not 3
```

Q&A

Q: Why `await` before calling an async function? Calling an `async def` returns a *coroutine object* — paused, not yet run. `await` hands it to the event loop to actually execute. Without `await`, you get `<coroutine object …>` and nothing runs.

Q: What is an event loop? The engine that drives async. Restaurant manager analogy: instead of standing at one table waiting on the kitchen, the manager circulates between tables and delivers orders when ready. Jupyter has a loop already running, so use `await main()` directly; standalone scripts use `asyncio.run(main())`.

Q: Why does `asyncio.run(main())` fail in Jupyter? Jupyter already has an event loop running. `asyncio.run` tries to start a new one — you can't nest event loops. In notebooks, use top-level `await`.

1.4 Tooling: `uv`

Modern Python package manager. 10-100x faster than pip; handles venvs automatically.

```
curl -LsSf https://astral.sh/uv/install.sh | sh
uv init my-project
uv add pydantic httpx anthropic
uv run python main.py
```

1.5 Week 1 Mini-Project: Markdown Notes Indexer

What it does

Takes a folder of `.md` files, extracts headings, word count, and last-modified time per file, processes them in parallel, outputs a JSON summary.

Concepts combined

Type hints • Pydantic model • `pathlib` • async with `aiofiles` • `asyncio.gather` • `json.dumps` with `default=str`

Final code

```
from datetime import datetime
from pathlib import Path
from pydantic import BaseModel
import aiofiles
import asyncio
import json

class MarkDownIndexer(BaseModel):
    path: str
    headings: list[str]
    word_count: int
    last_modified: datetime

async def summarize_note(path: Path) -> MarkDownIndexer:
    async with aiofiles.open(path, mode="r") as f:
        content = await f.read()
    last_modified = datetime.fromtimestamp(path.stat().st_mtime)
    word_count = len(content.split())
    lines = content.splitlines()
    headings = []
    for line in lines:
        if line.startswith("#"):
            headings.append(line.lstrip("# "))
```

```
    return MarkDownIndexer(
        path=str(path),
        headings=headings,
        word_count=word_count,
        last_modified=last_modified,
    )

async def main(directory: Path) -> list[MarkDownIndexer]:
    files = list(directory.glob("**/*.md"))
    return await asyncio.gather(*[summarize_note(f) for f in files])

# Output
result = await main(Path("./sample_notes"))
for_json = [i.model_dump() for i in result]
print(json.dumps(for_json, default=str, indent=2))
```

Key things learned

- **Path object is not str** — Pydantic can't serialize `Path` natively, store as `str(path)`
- **`* unpacks lists, ** unpacks dicts** — asyncio.gather(*tasks) vs func(**kwargs)`
- **`model_dump()` keeps Python types** — `datetime` stays a `datetime` object. `json.dumps()` converts to string. Use `default=str` to tell json.dumps to coerce unknown types to strings.
- **List comprehensions** are the Pythonic way to build a list from a loop in one line.

Pitfalls hit

- Mutable default arguments — use `None` and check inside
- Calling async function without `await` — gives coroutine object, not result
- Building wrong path — used `/Phase 1/…` instead of `/Volumes/MainDrive/ai engineering practice/Phase 1/…`. Use a base `Path` and `/` operator instead

---

Week 2: APIs & LLM Basics

2.1 The Three Providers

All three use the same `messages = [{"role": ..., "content": ...}]` format. Differences:

Provider	Client	Endpoint method	Text path
Anthropic	`Anthropic()`	`messages.create()`	`response.content[0].tex`
OpenAI	`OpenAI()`	`chat.completions.create()`	`response.choices[0].mes`

LM Studio (local)	`OpenAI(base_url="http://localhost:1234/v1", api_key="not-needed")`	same as OpenAI	same

Why `localhost:1234/v1`?

LM Studio exposes an OpenAI-compatible API. The `/v1` is the OpenAI URL convention — required so the SDK's path-building works unchanged.

Why mention local LM in interviews?

> "I prototype against my local 20B model first to iterate fast and stay private, then swap to a hosted model for production." — senior signal.

2.2 Loading secrets

```
# .env
ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-...
```

```
from dotenv import load_dotenv
import os

load_dotenv(Path("/full/path/.env"))  # only needed if .env isn't in cwd
key = os.getenv("ANTHROPIC_API_KEY")
```

Why env vars

Code gets shared (GitHub, Stack Overflow, colleagues). Hardcoded keys leak. Keep secrets out of code; load from environment.

`.gitignore` essentials

`.env`, `*.db`, anything containing secrets or large local data.

2.3 Three prompting patterns

1. System prompt

Sets persona/rules. Separate `system=` parameter on Anthropic; first message with `"role": "system"` on OpenAI.

2. Few-shot

Show examples of input → expected output before the real question. Model infers the pattern. Especially good when zero-shot is unreliable on a specific format.

```
messages = [
    {"role": "user", "content": "Movie was really bad."},
    {"role": "assistant", "content": "Negative"},
    {"role": "user", "content": "Movie was awesome."},
    {"role": "assistant", "content": "Positive"},
    {"role": "user", "content": "I kind of liked the movie."},  # real query
]
```

3. Structured outputs (OpenAI)

Force JSON conforming to a Pydantic schema.

```
class Sentiment(BaseModel):
    label: str
    confidence: float = Field(ge=0, le=1)
    reasoning: str

response = openai_client.chat.completions.parse(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "I loved the movie!"}],
    response_format=Sentiment,
)
result: Sentiment = response.choices[0].message.parsed
```

Q&A

Q: How does structured output actually work? The SDK serialises your Pydantic model to a JSON schema and ships it with the request. The provider constrains generation to match. The SDK then runs `Model.model_validate_json()` on the JSON string and returns a typed object via `response.choices[0].message.parsed`.

2.4 FastAPI

What it is

Python web framework for APIs. Built on Pydantic + async. Auto-generates Swagger docs at `/docs`.

Server vs notebook

A notebook runs cells top-to-bottom and stops. A server runs continuously, listening for requests. Servers need a `.py` file and run from terminal (`uv run fastapi dev main.py` or `uv run uvicorn`

`main:app --reload`).

Minimal app

```
from fastapi import FastAPI
from pydantic import BaseModel
from anthropic import Anthropic

app = FastAPI()
client = Anthropic()  # one client, reused across requests

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str
    tokens_used: int

@app.get("/health")
async def health_check():
    return {"status": "ok"}

@app.post("/chat")
async def chat(req: ChatRequest) -> ChatResponse:
    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=1000,
        messages=[{"role": "user", "content": req.message}],
    )
    return ChatResponse(
        reply=response.content[0].text,
        tokens_used=response.usage.input_tokens + response.usage.output_tokens,
    )
```

Pitfalls hit

- `fastapi` CLI requires `fastapi[standard]` — `uv add "fastapi[standard]"`
- Defining the client inside the route function recreates it per request — define at module level
- Mixing `BaseModel` and `TypedDict` as base classes is wrong — use only `BaseModel` for FastAPI

2.5 Week 2 Mini-Project: Unified LLM Wrapper

Goal

One async `complete()` function for Anthropic / OpenAI / local, plus SQLite logging, cost calculation, and latency tracking. Foundation for every later project.

Signature

```
async def complete(
    prompt: str,
    provider: Literal["openai", "anthropic", "local"],
    model: str | None = None,
    system: str | None = None,
    response_schema: type[BaseModel] | None = None,
) -> CompletionResult:
    ...
```

Return model

```
class CompletionResult(BaseModel):
    text: str
    parsed: BaseModel | None
    input_tokens: int
    output_tokens: int
    cost_usd: float
    latency_ms: int
    provider: str
    model: str
```

Key techniques

- **Default model per provider** via dict lookup (`defaults[provider]`)
- **Conditional kwargs** — build a `params` dict, add `system` only if not `None`, unpack with `**`
- **`time.perf_counter()`** before and after; multiply diff by 1000, wrap in `int()`
- **Price table** keyed by model; cost = `(in_tokens/1000)*p_in + (out_tokens/1000)*p_out`
- **OpenAI vs Anthropic system prompt** — OpenAI puts it in the messages list with `role="system"` (use `.insert(0, …)`); Anthropic uses a top-level `system=` parameter
- **`match/case`** for provider branching (Python 3.10+)
- **SQLite logging via SQLModel** — table class with `table=True`, autocommit per call inside `with Session(engine) as session: …`

CallLog table

```
class CallLog(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    prompt: str
    provider: str
    model: str
    input_tokens: int
    output_tokens: int
    cost_usd: float
    latency_ms: int
    text: str
    timestamp: datetime = Field(default_factory=datetime.now)
```

Q&A from the build

Q: Why is `id` typed `int | None` if primary keys are never null? Before insertion, Python doesn't yet know the auto-generated ID — Pydantic would complain if `id` were required. After insertion, the DB fills it in. Optional in Python perspective, never null in DB perspective.

Q: SQL vs SQLite? SQL is a language. SQLite is one database engine that speaks SQL. SQLite is serverless — the whole DB lives in a single file (`llm_logs.db`). No service to start, no credentials, no setup. Production uses PostgreSQL/MySQL; portfolio projects use SQLite.

Q: Why `with Session(engine) as session:`? `with` guarantees the session closes even on exceptions. Same reason files use `async with aiofiles.open(...)`. Without it: connection leaks if something throws between `add` and `close`.

Q: Why `add` then `commit` (two steps)? You can `add` multiple objects and commit as one transaction. If anything fails mid-way, nothing gets written — consistency guarantee.

Q: Module reload in Jupyter? Once Python imports a module, it caches it. Editing the file doesn't reload it in the same kernel. Use `importlib.reload(llm_wrapper)` then re-import.

Test result (anthropic)

```
CompletionResult(
    text='RAG (Retrieval-Augmented Generation) combines a retrieval system that fetches relev
    parsed=None,
    input_tokens=15,
    output_tokens=45,
    cost_usd=0.00072,
    latency_ms=2833,
    provider='anthropic',
    model='claude-sonnet-4-6',
)
```

Claude ~27× more expensive than gpt-4o-mini per token for this call → why local matters for prototyping.

Week 3: Embeddings & Vector Search

3.1 Embeddings — what they are

A fixed-length vector of floats representing the *meaning* of a piece of text. Similar meaning → similar vectors. Length depends on the model (1536 for OpenAI `text-embedding-3-small`, 384 for `bge-small-en-v1.5`).

Fixed size for any input length

The Transformer reads all tokens together, then pools them (average / special summary token) into

one fixed-size vector. Regardless of input length, output dimension is constant — that's what enables similarity comparison.

Multi-token words

A word like "embeddings" may be 2+ tokens. Each token has an initial vector; attention updates each token's vector based on surrounding tokens; pooling collapses them into one. So multi-token words still get one combined vector.

Why context matters

"bank" alone is ambiguous (river / financial). In context, attention shifts the meaning. Same word in different sentences produces different vectors.

3.2 Cosine similarity

Measures the **angle** between two vectors, not the distance. Range –1 (opposite) to 1 (identical). For text embeddings, typically 0 to 1.

```
import numpy as np

def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

`np.linalg.norm(a)` = vector magnitude (Pythagoras extended to N dimensions: `sqrt(x1² + x2² + … + xN²)`).

Numbers seen

Pair	OpenAI 1536d	BGE local 384d
dog vs puppy	0.56	0.87
dog vs refrigerator	0.22	0.59

Same ranking; BGE produces consistently higher absolute scores. Always compare to itself — never mix embeddings across models.

3.3 Qdrant

What it is

A vector database. Stores vectors with payloads and offers fast nearest-neighbour search via HNSW

index. Open source. Runs locally via Docker.

Normal DB vs vector DB

Normal DB (SQLite)	Qdrant
Table	Collection
Row	Point
Column values	Payload
Primary key	Point ID
Index (B-tree)	Index (HNSW)
`WHERE name = 'X'`	`query_vector = embedding`
Exact match	Nearest neighbour search

Docker setup

```
docker run -p 6333:6333 -p 6334:6334 \
  -v "$(pwd)/qdrant_storage:/qdrant/storage" \
  qdrant/qdrant
```

- `-p HOST:CONTAINER` — maps host port to container port
- `-v HOST_DIR:CONTAINER_DIR` — persists data on host so it survives container restarts
- Without volume mount, data disappears when container is removed

Code usage

```
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct

client = QdrantClient(url="http://localhost:6333")

client.create_collection(
    collection_name="words",
    vectors_config=VectorParams(size=1536, distance=Distance.COSINE),
)

client.upsert(
    collection_name="words",
    points=[
        PointStruct(id=1, vector=emb_dog, payload={"text": "dog"}),
        PointStruct(id=2, vector=emb_puppy, payload={"text": "puppy"}),
    ],
)

results = client.query_points(           # newer API; replaces .search()
```

```
    collection_name="words",
    query=query_embedding,                # plain list[float]; not the OpenAI response object
    limit=5,
)
for p in results.points:
    print(p.score, p.payload["text"])
```

Pitfalls hit

- `search()` is deprecated — use `query_points()` with `query=` (not `query_vector=`)
- Pass plain `list[float]`, not the OpenAI response object — extract via `response.data[0].embedding`
- Point IDs must be integers (or UUIDs), not strings
- Without volume mount, the DB starts empty after every container restart

Q&A

Q: Where are my embeddings stored on disk? Inside the mounted `qdrant_storage` folder. They're in binary segment files optimized for HNSW search, not human-readable. To inspect, visit the dashboard at `http://localhost:6333/dashboard`.

3.4 Week 3 Mini-Project: Semantic Search CLI

Built with Typer. Two subcommands:

Index command

```
@app.command()
def index(directory: str):
    content = []
    file_names = []
    for file in Path(directory).glob("**/*.md"):
        content.append(file.read_text())
        file_names.append(file)
    embedding = client.embeddings.create(
        model="text-embedding-3-small", input=content,
    )
    client_qdrant.delete_collection("my-docs")
    client_qdrant.create_collection(
        collection_name="my-docs",
        vectors_config=VectorParams(
            size=len(embedding.data[0].embedding),
            distance=Distance.COSINE,
        ),
    )
    points = []
    for i, file in enumerate(file_names):
        points.append(PointStruct(
```

```
            id=i,
            vector=embedding.data[i].embedding,
            payload={"text": content[i][:200], "filename": file.name},
        ))
    client_qdrant.upsert(collection_name="my-docs", points=points)
    print(f"Indexed {len(file_names)} files.")
```

Query command

```
@app.command()
def query(search_query: str):
    print(f"Querying for {search_query}")
    search_embedding = client.embeddings.create(
        model="text-embedding-3-small", input=search_query,
    )
    search_result = client_qdrant.query_points(
        collection_name="my-docs",
        query=search_embedding.data[0].embedding,
        limit=5,
    )
    for point in search_result.points:
        print(point.score, point.payload["filename"])
```

Sample run

```
$ uv run python search.py query "how do embeddings work"
0.606713 embeddings_basics.md
0.3864751 vector_databases.md
0.27255446 rag_overview.md
```

Why OpenAI embeddings over BGE for this corpus

Documents are fairly long and cover diverse topics (embeddings, RAG, vector DBs). Higher dimensionality (1536 vs 384) gives the model more room to separate related-but-distinct concepts. BGE would shine on a more focused corpus where 384 dimensions suffice.

Week 4: RAG (Retrieval-Augmented Generation)

4.1 What RAG is

```
prompt → retrieve relevant docs → stuff into prompt → LLM → grounded answer
```

LLM training data is frozen and public. RAG plugs in your *current, private, or specialized* data at query time.

Two steps:

1. **Retrieval** — embed query, search vector DB, return top-K relevant chunks
2. **Generation** — pass chunks to LLM as context, instruct it to answer using only the context

---

4.2 Chunking

Why chunk

- LLM context windows are limited; you can't fit a 500-page doc in a prompt
- Cost — bigger prompts = more tokens
- **Retrieval quality** — a single vector for a long doc averages all meanings; chunked vectors are focused. A query about one section matches the right chunk instead of the diluted whole-doc vector.

Four strategies

Strategy	What it does	When to use
Fixed-size	Split every N chars/tokens	Repetitive content (logs, transcripts)
Recursive	Try `\n\n` → `\n` → `.` → → `""`	Default for most documents
Semantic	Split where topic changes (embedding similarity between sentences)	Very heterogeneous content; usually overkill
Structure-aware	By sections/functions for MD/HTML/code	Technical docs, code

How `RecursiveCharacterTextSplitter` works

Tries the first separator. If chunks are still too big, splits the oversized ones with the next separator. Recurses until everything fits under `chunk_size`. Adds `chunk_overlap` characters of the previous chunk to the next — preserves continuity at boundaries.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
    separators=["\n\n", "\n", ". ", " ", ""],
)
chunks = splitter.split_text(text)
```

4.3 GDPR RAG: ingestion

Loading the source

Initially tried PDF extraction with `pypdf` — produced garbled output (`"Legislativ e"`, `"Apr il"`). Switched to copy-pasting clean HTML from EUR-Lex into `gdpr.txt`. Resulting text: 348,149 characters → 1,108 chunks at chunk_size 500.

Batching for embedding

OpenAI API can't take all chunks at once. Split into batches of 100:

```
batch_chunks = []
temp_batch = []
for i in range(len(chunks)):
    temp_batch.append(chunks[i])
    if (i + 1) % 100 == 0:
        batch_chunks.append(temp_batch)
        temp_batch = []
if temp_batch:
    batch_chunks.append(temp_batch)
```

Ingestion loop with running ID counter

```
client_qdrant.delete_collection("gdpr")
client_qdrant.create_collection(
    collection_name="gdpr",
    vectors_config=VectorParams(size=1536, distance=Distance.COSINE),
)
start_id = 0
for chunk in batch_chunks:
    response = client.embeddings.create(
        model="text-embedding-3-small", input=chunk,
    )
    embeddings = [e.embedding for e in response.data]
    points = []
    for i in range(len(chunk)):
        start_id += 1
        points.append(PointStruct(
            id=start_id,
            vector=embeddings[i],
            payload={"text": chunk[i]},
        ))
    client_qdrant.upsert(collection_name="gdpr", points=points)
```

Pitfalls hit

- Reset `id=i` per batch → collision and overwrite. Fix: running counter

- Truncating payload to `[:200]` → lost text needed for prompt. Fix: store full chunk
- Creating collection inside the loop → recreated every batch. Fix: outside

---

4.4 GDPR RAG: query

```
def query(prompt: str, client: OpenAI, client_qdrant: QdrantClient) -> str:
    search_embedding = client.embeddings.create(
        model="text-embedding-3-small", input=prompt,
    )
    search_result = client_qdrant.query_points(
        collection_name="gdpr",
        query=search_embedding.data[0].embedding,
        limit=5,
    )
    context = "\n\n".join(
        f"[Chunk {i+1}]: {point.payload['text']}"
        for i, point in enumerate(search_result.points)
    )
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": (
                "Answer based only on the provided context. "
                "Cite your sources using [Chunk N] references. "
                "If the context doesn't contain the answer, say "
                "'I don't know based on the provided context.' "
                "Do not make up information.\n\n"
                f"{context}"
            )},
            {"role": "user", "content": prompt},
        ],
    )
    return response.choices[0].message.content
```

The two non-negotiables in the prompt

1. **Anti-hallucination** — "say 'I don't know' if not in context"
2. **Citations** — `[Chunk N]` references in the answer so you can verify

Sample answer (verified)

> Data subjects have the following rights:
>
> 1. Right of access to personal data … [Chunk 1][Chunk 2]
> 2. Right to rectification … [Chunk 3]
> 3. Right to erasure (right to be forgotten) … [Chunk 3]
> 4. Right to obtain confirmation of processing and access … [Chunk 5]

Off-topic test ("What is the capital of France?") → "I don't know based on the provided context." ✓ Anti-

hallucination works.

4.5 The evaluation problem

After a RAG pipeline runs end-to-end, the *real* question begins: **is it any good?** "Looks reasonable when I try it" is not a measurement. Without a metric, every later tuning decision (different chunking, different embedding model, reranking, etc.) is guesswork.

Two stages, two failure modes

RAG has two distinct stages and either one can fail. The fix is different in each case, so they need to be measured separately.

Stage	Failure looks like	Common metrics
Retrieval	Wrong chunks returned for the query	recall@k, precision@k, MRR, nDCG
Generation	Right chunks retrieved, but the answer hallucinates / drifts / dodges the question	RAG triad (see 4.10)

The discipline: separate them. If your final answer is bad, you must know whether retrieval brought you garbage or generation mishandled good context. The fixes are completely different.

The cold-start eval problem

The blocker most people hit: *I don't have labeled question/answer pairs for my corpus.* You don't need many. A small (20–50) hand-verified "golden set" is more useful than a large noisy one.

Two paths, and you should do both in order:

1. **Synthetic generation** — get moving fast
2. **Human curation** — quality control on the synthetic output

---

4.6 Building a synthetic eval set

The idea: hand each chunk to an LLM, ask it to produce one question whose answer lives in that chunk. You now have `(question, gold_chunk_id)` pairs — the chunk is your labeled-relevant document.

The build script

```
random.seed(42)
sample_indices = random.sample(range(len(chunks)), 30)
```

```
eval_dataset = []
for chunk_id in sample_indices:
    chunk_text = chunks[chunk_id]
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": (
                "You are a legal QA expert. Given a passage from the GDPR, "
                "generate exactly ONE specific, realistic question that a "
                "compliance officer might ask, whose answer is clearly "
                "contained in the passage. Return ONLY the question."
            )},
            {"role": "user", "content": chunk_text},
        ],
        temperature=0.7,
    )
    eval_dataset.append({
        "question": response.choices[0].message.content.strip(),
        "gold_chunk_id": chunk_id,
        "gold_chunk_text": chunk_text[:200],  # preview, makes debugging easy
    })
```

What makes a *good* synthetic question

- **Specific**, not topic-level ("Can I keep customer email addresses after they unsubscribe?" beats "What does GDPR say about email?")
- **Realistic** — what a real user/compliance officer might actually ask
- **Uniquely answerable from this chunk** — if many chunks could answer it, the gold label is arbitrary

Built-in biases of synthetic evals (must know)

- **Best-case bias** — every question was synthesized *from* a chunk, so the chunk obviously contains the answer. Your real users phrase things in ways your chunks weren't written to match. Synthetic recall is an **upper bound**, not an estimate of real-world recall.
- **Question-type bias** — the generator's default style tends toward factoid questions. Real users also ask multi-hop, comparative, and "is this allowed?" questions.
- **Circularity** — if you use GPT-4 to write the questions and GPT-4 to answer them in your RAG, you're measuring "can GPT-4 answer questions GPT-4 wrote." Not fatal; be aware.

The "I don't know" gap

A good RAG should *refuse* to answer when the corpus doesn't contain the answer. None of the synthetic questions test this. Add 3–5 hand-written questions that are *not* answerable from your corpus to measure false-positive rate later.

Libraries (know the names)

- **Ragas** — `TestsetGenerator` for synthetic eval set generation

- **LlamaIndex** — `generate_question_context_pairs`

4.7 Stable chunk IDs and the off-by-one bug

To compute recall, you need a stable way to identify "which chunk did retrieval bring back vs. which one was labeled relevant." Two different ID spaces can silently disagree:

1. **Index in the `chunks` list** at chunking time (0, 1, 2, …)
2. **Qdrant point ID** stored alongside the vector

If these don't match, every recall measurement is wrong by construction, and you'll waste hours blaming embeddings.

The bug hit during this week

Original ingestion code:

```
start_id = 0
for chunk in batch_chunks:
    ...
    for i in range(len(chunk)):
        start_id += 1                           # increment FIRST
        points.append(PointStruct(id=start_id, ...))   # then append
```

First chunk lands at Qdrant ID **1**, not 0. But the eval-set script labeled `gold_chunk_id` using `range(len(chunks))`, which is 0-indexed. The two index spaces were **off by exactly one**. Every gold label pointed at the *previous* chunk's text in Qdrant. Recall would have looked near-zero, blamed on retrieval, fix would have been hunted in the wrong place.

The fix

```
for i in range(len(chunk)):
    points.append(PointStruct(id=start_id, ...))   # append first
    start_id += 1                                   # then increment
```

Now `chunks[i]` lands at Qdrant ID `i`. Aligned.

Verification discipline

"Logic looks right" ≠ "it is right." Verify by spot-checking:

- Pick 2–3 gold IDs from the eval JSON (a low, mid, and high one)
- Call `client_qdrant.retrieve(collection_name="gdpr", ids=[gold_id])`
- Confirm the returned payload text matches the `gold_chunk_text` preview saved in the eval

The structural fragility (parked until Phase 2)

Two scripts (the notebook and `build_eval.py`) chunk the same file *independently*. They happen to agree because the splitter is deterministic and both use identical params. Any divergence (lib bump, different params, different file read) silently breaks alignment. In production: have one script own chunking, write `(id → text)` to disk, both indexing and eval-gen read from that. Phase 1 doesn't require this.

The lesson

Correctness bugs that *silently corrupt results* are the dangerous kind. They don't crash; they make every downstream number meaningless. Spend the 5 minutes to verify ID alignment before running any eval.

4.8 Running retrieval eval — recall@k

Refactor query() to return IDs

For eval, the function that does retrieval has to surface the retrieved IDs, not just the final answer string:

```
def query(prompt: str, client: OpenAI, client_qdrant: QdrantClient) -> tuple[str, list[int]]:
    ...
    shortlisted_chunks = [point.id for point in search_result.points]
    ...
    return response.choices[0].message.content, shortlisted_chunks
```

Why tuple, not just string: retrieval eval cares about IDs, not the generated answer. The function under test has to expose both.

Don't duplicate retrieval in two places. If the eval loop reimplements `embed → query_points`, the eval drifts from what `query()` actually does the moment one changes. Have both call into the same retrieval primitive. (Skipped in this notebook for simplicity; production-relevant in Phase 2.)

The eval loop

```
hits = 0
for item in eval_data:
    question, gold_id = item["question"], item["gold_chunk_id"]
    emb = client.embeddings.create(model="text-embedding-3-small", input=question)
    result = client_qdrant.query_points(
        collection_name="gdpr",
        query=emb.data[0].embedding,
        limit=5,
    )
    retrieved_ids = [p.id for p in result.points]
```

```
    if gold_id in retrieved_ids:
        hits += 1
print(f"Recall@5: {hits}/{len(eval_data)} = {hits/len(eval_data):.1%}")
```

Result on GDPR

```
Recall@5: 26/30 = 86.7%
```

How to read this number

Two valid readings of the same number, both true:

- **Flattering**: "embeddings + cosine retrieval are pulling their weight"
- **Honest**: this is the *easy* eval — questions were synthesized FROM chunks. Real-world recall is the floor, not what you just measured. **86.7% is your ceiling, not your typical performance.**

Sanity check: predict before measuring

Random retrieval would give roughly `k / N` recall. For ~1100 chunks at k=5, that's ~0.5%. So 86.7% means embeddings are doing real work (>170× random). The "good" zone for naive RAG on a clean corpus is 60–80%. Above that = either a strong setup or an easy eval. (In this case, both.)

Rank-sensitive metrics (next step)

Recall@k is binary: hit or miss, regardless of position. A gold chunk at rank 1 vs. rank 5 both score "hit" but are not equally good retrievals.

MRR (Mean Reciprocal Rank) captures this. For each query, score = `1 / rank_of_first_gold_chunk` (or 0 if not in top-k). Average across queries. MRR rewards bringing the right chunk to the *top* of the list. Use when ranking matters (which is almost always for downstream generation quality).

4.9 Error analysis on the 4 failures

The aggregate metric (86.7%) tells you there are 4 things to investigate. The *failures* are where the actual learning lives. Reading them one by one:

Failure categories

Each retrieval miss falls into one of three buckets, and the fix differs by bucket:

Category	Symptom	Fix

Eval-set problem	Question is ambiguous, fabricated, or over-broad	Fix the eval (cull/rewrite questions)
Retrieval problem	Right idea, wrong chunk pulled	Hybrid search, reranking, better chunking
Vocabulary mismatch	Question and chunk use different words for the same concept	HyDE, query rewriting

The 4 actual misses (GDPR eval)

Miss 1 — chunk 189 (`"(62)"`) The chunk is just a recital number marker. The generator LLM, given essentially empty content, **fabricated** a generic question about controller obligations that the chunk doesn't actually answer. **Category: eval-set problem.** Root cause: chunking produced stub chunks (the recursive splitter isolated single recital markers between blank lines), and the generation prompt didn't filter them out.

Miss 2 — chunk 476 (`"(150)"`) Same pattern as miss 1. Stub chunk → fabricated question about "data protection by design and by default." **Category: eval-set problem.** Two failures with the same root cause = systematic flaw, not noise.

Miss 3 — chunk 447 ("...the right to a judicial remedy on behalf of data subjects...") Chunk is real GDPR Article 80 content about *representative bodies* mandated by data subjects. The generator hallucinated a question about **"compliance officer"** — a completely different role that appears nowhere in this chunk. Retrieval correctly returned compliance-officer-related chunks (DPO duties etc.), none of which were 447. **Category: eval-set problem (misattribution) with a category-3 flavor** — the question's key noun doesn't appear in the chunk, exactly the vocabulary-mismatch case HyDE addresses.

Miss 4 — chunk 13 (Recital 6, "Technology has transformed...") Recital 6 is a high-level preamble that *mentions* third-country transfers as a topic. The generated question — "How does GDPR aim to ensure protection during transfer to third countries?" — is answered far more directly by chunks from **Chapter V** (adequacy decisions, SCCs, BCRs). Retrieval correctly returned the better-answering chunks. **Category: over-broad question.** The synthesizer wrote a *topic* question, not a *chunk-specific* question.

The verdict

All 4 misses are eval-set problems. None are retrieval failures. Real recall@5 on the *culled* version of this eval ≈ 100%. The system isn't broken; the eval has bugs.

The two takeaways

1. **A metric is a starting point for investigation, not a conclusion.** People who skip error analysis end up "fixing" non-problems or shipping systems that look great on numbers but fail real users.
2. **Synthetic evals have systematic failure modes you now recognise:** stub chunks → fabricated questions, and over-broad topic questions → multiple valid answers, only one labeled

gold.

4.10 Generation eval — the RAG triad

Retrieval eval covers stage 1. Even with *perfect* retrieval, generation can still fail in three distinct ways. The **RAG triad** (framing from TruLens; metrics implemented in Ragas, DeepEval, etc.) measures each.

Metric	Question it asks	Failure mode it catches
Context relevance	Of the chunks retrieved, what fraction are actually relevant to the question?	Noisy top-k, gold chunk buried in garbage
Faithfulness (groundedness)	Of the claims in the answer, how many are supported by the retrieved context?	**Hallucination** — model fabricates facts not in context
Answer relevance	Does the answer actually address the question?	Drift, summarising the context instead of answering, dodging

Context relevance ≠ recall@k

Easy to conflate; they are different:

- **Recall@k** asks: "Was the labeled-gold chunk in the top-k?" (presence/absence of one specific chunk)
- **Context relevance** asks: "What fraction of the top-k was actually relevant?" (noise level)

You can have 100% recall with 20% context relevance (right chunk + 4 irrelevant ones). You can have 0% recall with 80% context relevance (the *labeled* gold was missed, but the 4 chunks retrieved are all reasonable answers). Both signals matter.

The faithfulness sleeper risk in known domains

Counterintuitive but important: a model is MORE likely to produce undetectable hallucinations on a domain it knows well from pretraining (GDPR, common law, popular APIs) than on an obscure one.

Why? On a known domain, the model has the facts baked in. When the retrieved context is incomplete, it confidently fills the gap with prior knowledge — facts that are often *correct in general* but **not from the context**. They sound right, so you don't catch them by reading. Faithfulness eval mechanically checks "is every claim in this answer entailed by the retrieved chunks?" — exactly the check humans skip.

On an obscure domain (internal company docs, niche specs), hallucinations are obviously wrong and easy to spot manually.

Domain	Hallucination frequency	Detectability by reading	Faithfulness eval value

Known to model (GDPR, etc.)	Medium	**Low** (sounds right)	**High**
Unknown to model	Higher	High (obviously wrong)	Lower

Reading order for the triad

Compute order: parallel, doesn't matter. **Reading order:** pipeline-flow (context relevance → faithfulness → answer relevance) so upstream signals gate the meaning of downstream ones. If context is garbage, faithfulness is hard to interpret. If faithfulness is broken, answer relevance is hard to interpret.

Cost ordering (useful when budget-constrained)

- **Answer relevance** — cheapest: one LLM-judge call per `(Q, A)` pair
- **Context relevance** — moderate: one call per retrieved chunk per question (k× per question)
- **Faithfulness** — most expensive: decompose the answer into atomic claims (1 call), verify each claim against context (1 call per claim). Easily 5–10× the cost of answer relevance.

4.11 LLM-as-judge

The key trick that makes all generation eval possible: there is **no ground-truth answer** to compare against. You don't have a "correct answer" labeled for each question. So you use an LLM (typically GPT-4-class, a *stronger* model than the one generating) as the grader.

For faithfulness: give the judge `(answer, context)`, ask *"is every claim in this answer supported by this context?"* For answer relevance: give the judge `(question, answer)`, ask *"does this answer address this question?"* The judge returns a score (0–1, or yes/no per claim, depending on framework).

Why it works

Evaluation is *easier* than generation. Asking "is X supported by Y?" requires verification, not authorship. Verification < generation in difficulty for an LLM.

Why it's imperfect (be honest about this)

- Judges have biases: prefer longer answers, prefer their own writing style, prefer confident wording even when wrong
- Different judge models give different scores on the same data
- The judge can be wrong; spot-check against human labels on a sample
- Same-family judges are systematically more lenient on same-family generators (use a different family if you can)

Libraries to know

- **Ragas** — RAG-specific, packages the triad + correctness + similarity metrics
- **TruLens** — origin of the "RAG triad" framing; observability-flavored
- **DeepEval** — newer, broader test-runner style

LLM-as-judge is the dominant paradigm in modern eval — RAG, agents, chatbots, alignment research. Internalise it now; it shows up everywhere in Phase 2.

4.12 The RAG improvement menu (concept level)

Don't pick improvements by their coolness. Pick by what your error analysis surfaces. Each fix targets a specific failure mode.

1. Hybrid search — fixes "rare exact term" misses

Dense embeddings are bad at exact-token matches on rare terms (article numbers, IDs, error codes, "Article 17"). They encode *semantic* similarity, so a chunk about "the right to be forgotten" can outrank the chunk literally containing the string "Article 17."

The fix: run two retrievers in parallel — dense (embedding) AND sparse (BM25-style keyword). Combine top-k from each. BM25 nails exact-term matches; dense nails paraphrase/synonym. They cover each other's weaknesses.

When to reach for it: any corpus with formal identifiers — legal articles, error codes, drug names, ticker symbols, product names. GDPR is textbook.

Library: Qdrant supports hybrid search natively via sparse vectors. Older stacks combined Elasticsearch (BM25) + vector DB manually.

2. Reranking — fixes "noisy top-k"

Initial retrieval pulls 5 chunks: 2 gold, 3 tangential noise. Or: the right chunk is at rank 8 and falls outside top 5.

The fix: retrieve a *larger* candidate set (top 20–50) with cheap embedding search. Then run a more expensive **cross-encoder** model over `(query, chunk)` pairs to re-rank. Cross-encoder reads query and chunk **jointly** with full attention — more accurate but too slow to run on the whole corpus. Use it as a precision filter on the top-N candidates.

Why cross-encoders beat bi-encoders: a bi-encoder embeds query and chunk separately, so it never sees them together. Cross-encoders attend over both at once — strictly more information available to the model.

When to reach for it: almost always once you care about quality. Highest-ROI improvement after basic retrieval works.

Libraries: Cohere Rerank API (commercial, easy), `cross-encoder/ms-marco-MiniLM` (HuggingFace, free, local), BGE rerankers.

3. HyDE — fixes "vocabulary mismatch"

Questions and source documents are written in different styles. User asks colloquially: *"Can I keep customer emails after they unsubscribe?"* The relevant chunk is formal: *"Personal data shall be erased without undue delay where the data subject has withdrawn consent."* Same meaning, almost no overlapping vocabulary — embedding similarity sometimes misses.

The fix (genuinely clever): before retrieval, ask an LLM to *generate a hypothetical answer* to the question. The answer is probably wrong in specifics — that's fine. But it's written in the **vocabulary and register of a real document**. Embed *that fake answer*, not the original question. The hypothetical answer lives closer to real source documents in embedding space.

When to reach for it: corpora whose writing style differs sharply from how users phrase questions (legal, scientific, technical docs vs. natural-language queries). Avoid on FAQ-shaped corpora — questions there *do* look like the source.

4. Query rewriting / expansion / decomposition — fixes "the question itself is the problem"

Three related techniques:

- **Rewriting** — user's question is ambiguous; use an LLM to clean it up before embedding ("what about cookies tho" → "What does GDPR require for cookie consent?")
- **Expansion (multi-query)** — generate several paraphrases of the question, retrieve against each, union the results. Helps when no single phrasing hits the right vocab.
- **Decomposition** — break a multi-hop question into sub-questions, retrieve for each independently. Bridge from retrieval to *agentic* RAG.

When to reach for them: specific failure modes — vague questions (rewriting), single-phrasing-misses (expansion), multi-hop questions (decomposition).

5. Better chunking — fixes "the units are wrong"

You saw this firsthand: 500-char recursive chunks produced stub chunks like `"(62)"`. Strategies:

- **Semantic chunking** — split at topic boundaries (embedding-similarity between sentences). More coherent chunks; more expensive at index time.
- **Sentence-window retrieval** — index single sentences, return surrounding sentences on retrieval. Precise matching, rich context.
- **Parent-document retrieval** — index small chunks (precision) but return larger parent sections on retrieval (context). Often best of both.

When to reach for these: when error analysis shows answers split across chunks, mid-sentence cuts, stub chunks polluting retrieval, or context too narrow for reasoning.

Decision framework

Failure pattern in error analysis	Reach for

Misses on exact-term queries (article numbers, IDs)	Hybrid search
Right chunk retrieved at low rank, or top-k is noisy	Reranking
Question and source use different vocabulary	HyDE
Vague, ambiguous, or multi-hop questions	Query rewriting / decomposition
Stub chunks, fragmented context	Better chunking
Hallucinations despite good retrieval	Tighter system prompt, smaller k, faithfulness eval

The loop that defines modern RAG work: **measure → analyse failures → pick the targeted fix → re-measure.** Not "stack all the techniques and hope."

4.13 Sparse vs dense vectors

Hybrid search assumes you understand both. Quick reference.

Mental model

	Dense embedding	**Sparse vector (BM25-style)**
What it encodes	Semantic similarity	Weighted exact-term overlap
Dimensionality	~hundreds–low thousands (e.g. 1536)	Vocabulary size (~tens of thousands+)
Density	All non-zero	Mostly zero
Strength	Synonyms, paraphrase, vocab mismatch	Rare proper nouns, exact phrases, IDs
Weakness	Rare/exact tokens	Synonyms, semantic paraphrase

Common misconception (corrected this week)

"Sparse = vector of 0s and 1s" — wrong. Non-zero values are **weighted scores**, not presence flags. Common weighting:

- **TF-IDF** = term frequency × inverse document frequency. Rare-but-frequent-here = high weight.
- **BM25** = TF-IDF refined with frequency saturation (10th occurrence counts much less than 2nd) and doc-length normalization. What most modern keyword retrievers use.

A chunk `"GDPR Article 17 requires erasure"` produces sparse weights roughly like:

- "gdpr": ~0.0 (everywhere → low weight)
- "article": ~0.0 (common)
- "17": ~2.4 (rare → high weight)
- "erasure": ~3.1 (rare → high weight)

- all other 50,000 dims: 0

The storage trick

Huge dimensionality doesn't blow up storage. Sparse vectors are stored as `(index, value)` pairs **only for non-zero entries** — typically dozens to a few hundred per document. The "50,000-dim sparse vector" is physically ~80 pairs on disk.

4.14 What was deferred — Phase 2 territory

Phase 1 is concept-level. The following were named but **not implemented**, and are left for Phase 2:

- Actually running Ragas (or equivalent) generation eval on the GDPR pipeline
- Implementing hybrid search with sparse + dense in Qdrant
- Implementing a reranker (Cohere or local cross-encoder)
- Comparing chunking strategies empirically (semantic vs recursive vs sentence-window)
- HyDE end-to-end
- Ablation studies — "removing X drops recall by N%"
- Production concerns: single source of truth for chunk IDs, structured eval harness, persisting per-question eval results

The Phase 1 mental rule: *does this affect whether I learn the concept correctly?* If yes, do it. If it's "this would be brittle in production" — note it, move on.

4.15 Week 4 — what to walk away with

- RAG has two stages (retrieval, generation) and they fail differently
- Recall@k is the basic retrieval metric; the RAG triad covers generation
- Synthetic evals have systematic biases (stub chunks → fake questions; over-broad questions → multiple valid answers)
- **Error analysis matters more than aggregate metrics** — the 4 GDPR failures taught more than the 86.7% number
- LLM-as-judge is how modern generation eval is actually done; understand its strengths and biases
- The improvement menu — know names, what each fixes, when to reach for each
- **Pick improvements by what error analysis shows, not by technique coolness**

Week 4 done at Phase-1 depth. Phase 1 paused at end of Week 4 to do an ML Sprint first; will return for Weeks 5 (agents) and 6 (evals).

Cross-cutting concepts

`with` statements

Context managers. Guarantee cleanup (close, release, commit) even on exceptions. Used for files, DB sessions, locks. Same shape as `async with` for async resources.

`*args` and `**kwargs` unpacking

- `*list_of_x` → spread items as positional args (e.g. `asyncio.gather(*tasks)`)
- `**dict_of_kv` → spread keys/values as keyword args (e.g. `func(**params)`)

`match/case`

Python 3.10+ pattern matching. Cleaner than chained `if/elif` for multi-way branching on a single value.

`enumerate()`

Iterate with index + value: `for i, x in enumerate(items): …`

`Literal[...]` type

Restricts a value to specific literals: `Literal["a", "b", "c"]` accepts only those three strings.

`type[BaseModel]`

A *class*, not an instance. Used when a parameter expects the class itself (e.g. `response_schema=Sentiment`).

`json.dumps(..., default=str)`

JSON has no `datetime` type. `default=str` says "if you don't know how to serialize something, call `str()` on it." Useful for serializing Pydantic-dumped dicts that still contain `datetime` objects.

Workflow rules picked up

- Always run `git init`, set up `.gitignore` before committing — never commit `.env`, `.db`, or large local data

- `git rm --cached path/to/file` stops tracking an already-committed file (file stays on disk)
- Module reload pattern in Jupyter: `importlib.reload(module); from module import x`
- Restart server after big changes; small ones auto-reload with `--reload`
- Use full absolute paths or `Path(base) / "subdir"` instead of relative strings to avoid working-directory bugs

---

Interview-ready talking points

- **Why type hints**: production Python is type-hinted by default; tooling catches bugs early; libraries (Pydantic, FastAPI) need them
- **Why Pydantic**: runtime validation at system boundaries; LLM outputs are untrusted JSON; coercion + clear error messages
- **Why async**: LLM calls are slow I/O; `asyncio.gather` runs N calls in the time of 1
- **Why local + hosted models**: prototype on local for speed/cost/privacy, ship on hosted for quality
- **Why log every LLM call**: cost tracking, latency analysis, debugging — Langfuse/Helicone in production, SQLite for portfolio
- **Why chunking matters**: small focused vectors retrieve precisely; overlap preserves continuity
- **Why anti-hallucination prompt + citations**: ungrounded answers and unverifiable answers both kill trust
- **Why evals are the moat**: anyone builds a RAG demo; only good engineers measure and improve it

---

End of notes. Last updated 2026-05-18.